

Name: Tooba Shafique

Reg no: 2023-BSE-065

Course: Cloud Computing Lab

Section: V-B

LAB 9

Codespaces + AWS: GH CLI (Codespaces), AWS CLI, EC2, IAM, Security Groups, Filters & Queries

Task 1 — GitHub CLI, Codespace setup and authentication

Goal: Install GH CLI (if not already present), authenticate with GitHub, create a Codespace, and connect to it. All work for the rest of the lab must be done inside the Codespace.

1. (Local desktop) Install GitHub CLI (Windows example via winget):

`winget install --id GitHub.cli`

```
C:\Users\malik>winget install --id GitHub.cli
The 'msstore' source requires that you view the following agreements before using.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
The source requires the current machine's 2-letter geographic region to be sent to the backend service
rly (ex. "US").

Do you agree to all the source agreements terms?
[Y] Yes [N] No: y
Found GitHub CLI [GitHub.cli] Version 2.83.2
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://github.com/cli/cli/releases/download/v2.83.2/gh_2.83.2_windows_amd64.msi
17.7 MB / 17.7 MB
Successfully verified installer hash
Starting package install...
Successfully installed
```

2. (Local) Authenticate GH CLI for Codespaces:

`gh auth login -s codespace`

When prompted, generate a GitHub Access Token (classic) in the browser with the following Token scopes: admin:org, codespace, repo.

Copy the token into the GH CLI prompt to complete login

```

C:\Users\malik>gh auth login -s codespace
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 28CA-4867
Press Enter to open https://github.com/login/device in your browser...
[?] Authentication complete.
- gh config set -h github.com git_protocol https
[?] Configured git protocol
[?] Logged in as toobashafique065

```

3. (Local) List available Codespaces (optional verification):

```
gh codespace list
```

```

C:\Users\malik>gh codespace list
no codespaces found

```

4. (Local) Create or connect to a Codespace. To create a new codespace from the current repo:

```
gh codespace create --repo <owner>/<repo> --branch main --machine basicLinux32gb
```

Or to open an existing codespace:

```
gh codespace ssh -c <name_of_codespace>
```

```
gh codespace ssh -c fuzzy-bassoon-4jp7p65jj46p37pwv
```

After connecting via `gh codespace ssh`, you will be inside a Linux shell that the rest of this lab assumes.

```

C:\Users\malik\Lab9>gh codespace create --repo toobashafique065/Lab9 --machine basicLinux32gb
[?] Codespaces usage for this repository is paid for by toobashafique065
fuzzy-bassoon-4jp7p65jj46p37pwv

```

```
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

@toobashafique065 [ ] /workspaces/Lab9 (main) $
```

Task 2 — Install AWS CLI inside the Codespace and configure it

Goal: Install the AWS CLI in the Codespace and configure it to interact with your AWS account.

Inside the Codespace shell run:

1. Download and install AWS CLI:

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
```

2. Verify installation:

```
aws --version
```

```
@toobashafique065 [ ] /workspaces/Lab9 (main) $ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left  Speed
100 60.3M  100 60.3M    0     0  194M      0  0:00:00  0:00:00  0:00:00  194M
@toobashafique065 [ ] /workspaces/Lab9 (main) $ unzip awscliv2.zip
Archive:  awscliv2.zip
  creating: aws/
  creating: aws/dist/
 inflating: aws/install
 inflating: aws/README.md
 inflating: aws/THIRD_PARTY_LICENSES
  creating: aws/dist/awscli/
  creating: aws/dist/dateutil/
  creating: aws/dist/docutils/
```

```
@toobashafique065 [ ] /workspaces/Lab9 (main) $ sudo ./aws/install
You can now run: /usr/local/bin/aws --version
@toobashafique065 [ ] /workspaces/Lab9 (main) $ aws --version
aws-cli/2.32.24 Python/3.13.11 Linux/6.8.0-1030-azure exe/x86_64.ubuntu.24
@toobashafique065 [ ] /workspaces/Lab9 (main) $
```

3. Configure the AWS CLI (you will provide Access Key ID and Secret Access Key for a user with permissions, or use root/IAM user you prepared for the lab):

```
aws configure
```

Enter:

AWS Access Key ID:

AWS Secret Access Key:

Default region name (e.g., me-central-1)

Default output format (e.g., json)

```
@toobashafique065 /workspaces/Lab9 (main) $ aws configure
AWS Access Key ID [None]: 
AWS Secret Access Key [None]: 
Default region name [None]: me-central-1
Default output format [None]: json
```

4. Verify credentials/config files:

`cat ~/.aws/credentials`

`cat ~/.aws/config`

```
@toobashafique065 /workspaces/Lab9 (main) $ cat ~/.aws/credentials
[default]
aws_access_key_id = 
aws_secret_access_key = 
@toobashafique065 /workspaces/Lab9 (main) $ cat ~/.aws/config
[default]
region = me-central-1
output = json
```

5. Verify connectivity (you should see a JSON result showing your caller identity):

`aws sts get-caller-identity`

```
@toobashafique065 /workspaces/Lab9 (main) $ aws sts get-caller-identity
{
  "UserId": "AIDAXZSFF03LIJ3H5VXDZ",
  "Account": "535942231766",
  "Arn": "arn:aws:iam::535942231766:user/LabUser"
}
```

Task 3 — Create security group and add ingress rules using Codespace IP

Goal: Create an EC2 security group, inspect it, add an SSH rule for your Codespace public IP, and verify.

Steps (run in the Codespace shell):

1. Create a security group (substitute your VPC id):

```
aws ec2 create-security-group --group-name 'MySecurityGroup' \
--description 'My Security Group' \
```

--vpc-id 'vpc-EXAMPLE1234567890'

```
@toobashafique065 [ ] /workspaces/Lab9 (main) $ aws ec2 create-security-group --group-name "MySecurityGroup" \
> --description "My Security Group" \
> --vpc-id "vpc-0b8fa1602193cc7f7"
{
  "GroupId": "sg-0f402fc7296bf67cb",
  "SecurityGroupArn": "arn:aws:ec2:me-central-1:535942231766:security-group/sg-0f402fc7296bf67cb"
}
@toobashafique065 [ ] /workspaces/Lab9 (main) $
```

2. Inspect the security group (initially ingress will be empty or default):

aws ec2 describe-security-groups --group-ids sg-EXAMPLE1234567890

```
@toobashafique065 [ ] /workspaces/Lab9 (main) $ aws ec2 describe-security-groups --group-ids sg-0f402fc7296bf67cb
{
  "SecurityGroups": [
    {
      "GroupId": "sg-0f402fc7296bf67cb",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ],
      "VpcId": "vpc-0b8fa1602193cc7f7",
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:535942231766:security-group/sg-0f402fc7296bf67cb",
      "OwnerId": "535942231766",
      "GroupName": "MySecurityGroup",
      "Description": "My Security Group",
      "IpPermissions": []
    }
  ]
}
```

3. Get your Codespace public IP (from inside the Codespace):

curl icanhazip.com

```
@toobashafique065 [ ] /workspaces/Lab9 (main) $ curl icanhazip.com
4.240.39.193
```

4. Authorize SSH inbound on port 22 from your Codespace IP:

aws ec2 authorize-security-group-ingress \

--group-id sg-EXAMPLE1234567890 \

--protocol tcp \

--port 22 \

--cidr <XXX.XXX.XXX.XXX>/32

"GroupId": "sg-0f402fc7296bf67cb"

Ip : 4.240.39.193

```
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 authorize-security-group-ingress \
> --group-id sg-0f402fc7296bf67cb \
> --protocol tcp \
> --port 22 \
> --cidr 4.240.39.193/32
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sgr-09ddf615c960e943b",
      "GroupId": "sg-0f402fc7296bf67cb",
      "GroupOwnerId": "535942231766",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 22,
      "ToPort": 22,
      "CidrIpv4": "4.240.39.193/32",
      "SecurityGroupRuleArn": "arn:aws:ec2:me-central-1:535942231766:security-group-rule/sgr-09ddf615c960e943b"
    }
  ]
}
```

5. Verify ingress rule appears:

aws ec2 describe-security-groups --group-ids sg-EXAMPLE1234567890

```
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 describe-security-groups --group-ids sg-0f402fc7296bf67cb
{
  "SecurityGroups": [
    {
      "GroupId": "sg-0f402fc7296bf67cb",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ],
      "VpcId": "vpc-0b8fa1602193cc7f7",
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:535942231766:security-group/sg-0f402fc7296bf67cb",
      "OwnerId": "535942231766",
      "GroupName": "MySecurityGroup",
      "Description": "My Security Group",
      "IpPermissions": [
        {
          "IpProtocol": "tcp",
          "FromPort": 22,
          "ToPort": 22,
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "4.240.39.193/32"
            }
          ]
        }
      ]
    }
  ]
}
```

6. Add an HTTP rule (port 80) using ip-permissions JSON:

aws ec2 authorize-security-group-ingress \

--group-id 'sg-EXAMPLE1234567890' \

--ip-permissions

'{"FromPort":80,"ToPort":80,"IpProtocol":"tcp","IpRanges":[{"CidrIp":"<XXX.XXX.XXX.XXX>/32"}]}'

```
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sgr-0dcc275d266510ae9",
      "GroupId": "sg-0f402fc7296bf67cb",
      "GroupOwnerId": "535942231766",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 80,
      "ToPort": 80,
      "CidrIpv4": "4.240.39.193/32",
      "SecurityGroupRuleArn": "arn:aws:ec2:me-central-1:535942231766:security-group-rule/sgr-0dcc275d266510ae9"
    }
  ]
}
```

7. Verify both ingress rules are present:

`aws ec2 describe-security-groups --group-ids sg-EXAMPLE1234567890`

```
@toobashafique065 [main] $ aws ec2 describe-security-groups --group-ids sg-0f402fc7296bf67cb | jq
{
  "SecurityGroups": [
    {
      "GroupId": "sg-0f402fc7296bf67cb",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ],
      "VpcId": "vpc-0b8fa1602193cc7f7",
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:535942231766:security-group/sg-0f402fc7296bf67cb",
      "OwnerId": "535942231766",
      "GroupName": "MySecurityGroup",
      "Description": "My Security Group",
      "IpPermissions": [
        {
          "IpProtocol": "tcp",
          "FromPort": 80,
          "ToPort": 80,
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "4.240.39.193/32"
            }
          ],
          "Ipv6Ranges": []
        }
      ]
    }
  ]
}
```

Task 4 — Create a key pair, describe key pairs, and launch EC2 instance

Goal: Create an ED25519 key pair, view it, and launch an EC2 instance using the key pair and the security group created earlier.

1. Create the key pair and save the PEM file into the Codespace workspace:

```
aws ec2 create-key-pair \
  --key-name MyED25519Key \
  --key-type ed25519 \
  --key-format pem \
```

```
--query 'KeyMaterial' \  
--output text > MyED25519Key.pem
```

```
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 create-key-pair \  
> --key-name MyED25519Key \  
> --key-type ed25519 \  
> --key-format pem \  
> --query 'KeyMaterial' \  
> --output text > MyED25519Key.pem
```

2. View created key pairs:

```
aws ec2 describe-key-pairs
```

```
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 describe-key-pairs  
{  
  "KeyPairs": [  
    {  
      "KeyPairId": "key-08db94fd170526b14",  
      "KeyType": "ed25519",  
      "Tags": [],  
      "CreateTime": "2025-12-27T13:10:42.729000+00:00",  
      "KeyName": "MyED25519Key",  
      "KeyFingerprint": "Wh8FkHdN27AhQ+RlZv4unYQhGdSpUt+zgWCubNJ1LXc="
```

3. (Do not) Delete key pair:

```
aws ec2 delete-key-pair --key-name MyED25519Key # Info: shows how to delete
```

4. Launch an EC2 instance (example values — replace IDs with ones from your account/region):

```
aws ec2 run-instances \  
--image-id ami-05e66df2bafcb7dea \  
--count 1 \  
--instance-type t3.micro \  
--key-name MyED25519Key \  
--security-group-ids sg-EXAMPLE1234567890 \  
--subnet-id subnet-EXAMPLE1234567890 \  
--tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=MyServer}]"
```

```
@toobashafique065 /workspaces/Lab9 (main) $ aws ec2 run-instances \
> --image-id ami-0f846b83494ddb084 \
> --count 1 \
> --instance-type t3.micro \
> --key-name MyED25519Key \
> --security-group-ids sg-0f402fc7296bf67cb \
> --subnet-id subnet-0d3e1389d968bb2ee \
> --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=MyServer}]"
{
  "ReservationId": "r-034ce9c8df290e0cf",
  "OwnerId": "535942231766",
  "Groups": [],
  "Instances": [
    {
      "Architecture": "x86_64",
      "BlockDeviceMappings": [],
      "ClientToken": "2e579ae6-9fd3-420a-acec-b7a76567b266",
      "EbsOptimized": false,
      "EnaSupport": true,
      "Hypervisor": "xen",
      "NetworkInterfaces": [
        {
          "Attachment": {
            "AttachTime": "2025-12-27T13:17:31+00:00",
            "AttachmentId": "eni-attach-0943ff68e9eebfafa",
            "DeleteOnTermination": true,
            "DeviceIndex": 0,
            "Status": "attaching",
            "NetworkCardIndex": 0
          },
          "Description": "",
          "Groups": [
            {
              "GroupId": "sg-0f402fc7296bf67cb",
              "GroupName": "MySecurityGroup"
            }
          ],
          "Ipv6Addresses": [],
          "MacAddress": "0a:82:a8:8d:83:8f",
          "NetworkInterfaceId": "eni-0871872b198d18644",
          "OwnerId": "535942231766",
          "PrivateDnsName": "ip-172-31-17-216.me-central-1.compute.internal",

```

5. Get the public IP address of your instance:

```
aws ec2 describe-instances \
--query "Reservations[*].Instances[*].[InstanceId,PublicIpAddress]" \
--output table
```

```
@toobashafique065 /workspaces/Lab9 (main) $ aws ec2 describe-instances \
> --query "Reservations[*].Instances[*].[InstanceId,PublicIpAddress]" \
> --output table
-----
| DescribeInstances |
|-----+-----|
| i-0bfe79bb9bc36dba8 | 40.172.195.252 |
|-----+-----|
@toobashafique065 /workspaces/Lab9 (main) $
```

6. Attempt SSH into the instance from the Codespace or from a machine whose IP is allowed in the security group:

```
ssh -i MyED25519Key.pem ec2-user@<public-ip-address>
```

If you see the error:

Permissions 0644 for 'MyED25519Key.pem' are too open. It is required that your private key files are NOT accessible by others.

fix permissions:

```
chmod 400 MyED25519Key.pem
```

```
ssh -i MyED25519Key.pem ec2-user@<public-ip-address>
```

```

@toobashafique065 @ /workspaces/Lab9 (main) $ ssh -i MyED25519Key.pem ec2-user@40.172.195.252
The authenticity of host '40.172.195.252 (40.172.195.252)' can't be established.
ED25519 key fingerprint is SHA256:3mTEG9wMlyBEH1o2v4SMyoOAyiV3od26KwxT66FpOec.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '40.172.195.252' (ED25519) to the list of known hosts.

#_
~\#### Amazon Linux 2
~~\#####
~~\###| AL2 End of Life is 2026-06-30.
~~\#/
~~V~'->
~~~~
~~~~ A newer version of Amazon Linux is available!
~~~~
~~~~ Amazon Linux 2023, GA and supported until 2028-03-15.
~/m/'- https://aws.amazon.com/linux/amazon-linux-2023/

```

7. Stop, start and (optionally) terminate the instance:

```
aws ec2 stop-instances --instance-ids i-EXAMPLE1234567890
```

```
aws ec2 start-instances --instance-ids i-EXAMPLE1234567890
```

```
aws ec2 terminate-instances --instance-ids i-EXAMPLE1234567890 # Don't run this command
```

```

@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 start-instances --instance-ids i-0bfe79bb9bc36dba8
{
  "StartingInstances": [
    {
      "InstanceId": "i-0bfe79bb9bc36dba8",
      "CurrentState": {
        "Code": 0,
        "Name": "pending"
      },
      "PreviousState": {
        "Code": 80,
        "Name": "stopped"
      }
    }
  ]
}
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 describe-instances \
> --instance-ids i-0bfe79bb9bc36dba8 \
> --query "Reservations[*].Instances[*].[InstanceId,State.Name,PublicIpAddress]" \
> --output table
-----
| DescribeInstances |
+-----+
| i-0bfe79bb9bc36dba8 | running | 3.28.208.136 |
+-----+
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 stop-instances --instance-ids i-0bfe79bb9bc36dba8
{
  "StoppingInstances": [
    {
      "InstanceId": "i-0bfe79bb9bc36dba8",
      "CurrentState": {
        "Code": 64,
        "Name": "stopping"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 start-instances --instance-ids i-0bfe79bb9bc36dba8
An error occurred (IncorrectInstanceState) when calling the StartInstances operation: The instance 'i-0bfe79bb9bc36dba8'
started.
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 describe-instances \
> --instance-ids i-0bfe79bb9bc36dba8 \
> --query "Reservations[*].Instances[*].[InstanceId,State.Name]" \
> --output table
-----
| DescribeInstances |
+-----+
| i-0bfe79bb9bc36dba8 | stopped |
+-----+

```

Task 5 — Understand AWS describe-* commands

Goal: Use describe commands to list and inspect AWS resources.

Run and understand these commands (run each, then capture screenshot immediately after):

`aws ec2 describe-security-groups`

```
@toobashafique065 @ /workspaces/Lab9 (main) $ aws ec2 describe-security-groups
{
  "SecurityGroups": [
    {
      "GroupId": "sg-0f402fc7296bf67cb",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ],
      "VpcId": "vpc-0b8fa1602193cc7f7",
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:535942231766:security-group/sg-0f402fc7296bf67cb",
      "OwnerId": "535942231766",
      "GroupName": "MySecurityGroup",
      "Description": "My Security Group",
      "IpPermissions": [
        {
          "IpProtocol": "tcp",
          "FromPort": 80,
          "ToPort": 80,
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "4.240.39.193/32"
            }
          ]
        }
      ]
    }
  ]
}
```

aws ec2 describe-vpcs

```
@toobashafique065 @ /workspaces/Lab9 (main) $ aws ec2 describe-vpcs
{
  "Vpcs": [
    {
      "OwnerId": "535942231766",
      "InstanceTenancy": "default",
      "CidrBlockAssociationSet": [
        {
          "AssociationId": "vpc-cidr-assoc-01aed097fc6b6afe4",
          "CidrBlock": "172.31.0.0/16",
          "CidrBlockState": {
            "State": "associated"
          }
        }
      ],
      "IsDefault": true,
      "BlockPublicAccessStates": {
        "InternetGatewayBlockMode": "off"
      },
      "VpcId": "vpc-0b8fa1602193cc7f7",
      "State": "available",
      "CidrBlock": "172.31.0.0/16",
      "DhcpOptionsId": "dopt-02c0a1e8f3dd284d6"
    }
  ]
}
```

aws ec2 describe-subnets

```
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 describe-subnets
{
  "Subnets": [
    {
      "AvailabilityZoneId": "mec1-az2",
      "MapCustomerOwnedIpOnLaunch": false,
      "OwnerId": "535942231766",
      "AssignIpv6AddressOnCreation": false,
      "Ipv6CidrBlockAssociationSet": [],
      "SubnetArn": "arn:aws:ec2:me-central-1:535942231766:subnet/subnet-0d3e1389d968bb2ee",
      "EnableDns64": false,
      "Ipv6Native": false,
      "PrivateDnsNameOptionsOnLaunch": {
        "HostnameType": "ip-name",
        "EnableResourceNameDnsARecord": false,
        "EnableResourceNameDnsAAAARecord": false
      },
      "BlockPublicAccessStates": {
        "InternetGatewayBlockMode": "off"
      },
      "SubnetId": "subnet-0d3e1389d968bb2ee",
      "State": "available",
      "VpcId": "vpc-0b8fa1602193cc7f7",
      "CidrBlock": "172.31.16.0/20",
      "AvailableIpAddressCount": 4090,
      "AvailabilityZone": "me-central-1b",
      "DefaultForAz": true,
      "MapPublicIpOnLaunch": true
    },
    {
      "AvailabilityZoneId": "mec1-az3",
      "MapCustomerOwnedIpOnLaunch": false,
      "OwnerId": "535942231766",

```

aws ec2 describe-instances

```
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 describe-instances
{
  "Reservations": [
    {
      "ReservationId": "r-034ce9c8df290e0cf",
      "OwnerId": "535942231766",
      "Groups": [],
      "Instances": [
        {
          "Architecture": "x86_64",
          "BlockDeviceMappings": [
            {
              "DeviceName": "/dev/xvda",
              "Ebs": {
                "AttachTime": "2025-12-27T13:17:32+00:00",
                "DeleteOnTermination": true,
                "Status": "attached",
                "VolumeId": "vol-044ed7b2bacd447a8"
              }
            }
          ],
          "ClientToken": "2e579ae6-9fd3-420a-acec-b7a76567b266",
          "EbsOptimized": false,
          "EnaSupport": true,
          "Hypervisor": "xen",
          "NetworkInterfaces": [
            {
              "Attachment": {
                "AttachTime": "2025-12-27T13:17:31+00:00",
                "AttachmentId": "eni-attach-0943ff68e9eebfafa",
                "DeleteOnTermination": true,
                "DeviceIndex": 0,
                "Status": "attached",

```

aws ec2 describe-regions

```
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 describe-regions
{
  "Regions": [
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "ap-south-1",
      "Endpoint": "ec2.ap-south-1.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "eu-north-1",
      "Endpoint": "ec2.eu-north-1.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "eu-west-3",
      "Endpoint": "ec2.eu-west-3.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "eu-west-2",
      "Endpoint": "ec2.eu-west-2.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "eu-west-1",
      "Endpoint": "ec2.eu-west-1.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "ap-northeast-3",
      "Endpoint": "ec2.ap-northeast-3.amazonaws.com"
    }
  ]
}
```

aws ec2 describe-availability-zones

```
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 describe-availability-zones
{
  "AvailabilityZones": [
    {
      "OptInStatus": "opt-in-not-required",
      "Messages": [],
      "RegionName": "me-central-1",
      "ZoneName": "me-central-1a",
      "ZoneId": "mec1-az1",
      "GroupName": "me-central-1-zg-1",
      "NetworkBorderGroup": "me-central-1",
      "ZoneType": "availability-zone",
      "GroupLongName": "Middle East (UAE) 1",
      "State": "available"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "Messages": [],
      "RegionName": "me-central-1",
      "ZoneName": "me-central-1b",
      "ZoneId": "mec1-az2",
      "GroupName": "me-central-1-zg-1",
      "NetworkBorderGroup": "me-central-1",
      "ZoneType": "availability-zone",
      "GroupLongName": "Middle East (UAE) 1",
      "State": "available"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "Messages": [],
      "RegionName": "me-central-1",
      "ZoneName": "me-central-1c",
      "ZoneId": "mec1-az3",
      "GroupName": "me-central-1-zg-1",
      "NetworkBorderGroup": "me-central-1",
      "ZoneType": "availability-zone",
      "GroupLongName": "Middle East (UAE) 1",
      "State": "available"
    }
  ]
}
```

Task 6 — IAM: create group, user, attach policies, create console login & keys

Goal: Practice managing IAM via CLI: create group and user, attach policies, add user to group, create login profile, attach/detach change-password policy, create access keys, and test access via environment variables.

Commands and immediate screenshot request after each step:

1. Create group: `aws iam create-group --group-name MyGroupCli`
2. Get group details: `aws iam get-group --group-name MyGroupCli`
3. Create user: `aws iam create-user --user-name MyUserCli`
4. Get user details: `aws iam get-user --user-name MyUserCli`

```
@toobashafique065 [~/workspaces/Lab9 (main)] $ aws iam create-group --group-name MyGroupCli
{
  "Group": {
    "Path": "/",
    "GroupName": "MyGroupCli",
    "GroupId": "AGPAXZSFF03LCQYEHNDLU",
    "Arn": "arn:aws:iam::535942231766:group/MyGroupCli",
    "CreateDate": "2025-12-27T14:26:04+00:00"
  }
}
@toobashafique065 [~/workspaces/Lab9 (main)] $ aws iam get-group --group-name MyGroupCli
{
  "Users": [],
  "Group": {
    "Path": "/",
    "GroupName": "MyGroupCli",
    "GroupId": "AGPAXZSFF03LCQYEHNDLU",
    "Arn": "arn:aws:iam::535942231766:group/MyGroupCli",
    "CreateDate": "2025-12-27T14:26:04+00:00"
  }
}
@toobashafique065 [~/workspaces/Lab9 (main)] $ aws iam create-user --user-name MyUserCli
{
  "User": {
    "Path": "/",
    "UserName": "MyUserCli",
    "UserId": "AIDAXZSFF03LFSOISHLCX",
    "Arn": "arn:aws:iam::535942231766:user/MyUserCli",
    "CreateDate": "2025-12-27T14:28:20+00:00"
  }
}
@toobashafique065 [~/workspaces/Lab9 (main)] $ aws iam get-user --user-name MyUserCli
{
  "User": {
    "Path": "/",
    "UserName": "MyUserCli",
    "UserId": "AIDAXZSFF03LFSOISHLCX",
    "Arn": "arn:aws:iam::535942231766:user/MyUserCli",
    "CreateDate": "2025-12-27T14:28:20+00:00"
  }
}
```

5. Add user to group:

`aws iam add-user-to-group --user-name MyUserCli --group-name MyGroupCli`

See group again:

`aws iam get-group --group-name MyGroupCli`

```
@toobashafique065 [~/workspaces/Lab9 (main)] $ aws iam add-user-to-group --user-name MyUserCli --group-name MyGroupCli
@toobashafique065 [~/workspaces/Lab9 (main)] $ aws iam get-group --group-name MyGroupCli
{
  "Users": [
    {
      "Path": "/",
      "UserName": "MyUserCli",
      "UserId": "AIDAXZSFF03LFSOISHLCX",
      "Arn": "arn:aws:iam::535942231766:user/MyUserCli",
      "CreateDate": "2025-12-27T14:28:20+00:00"
    }
  ],
  "Group": {
    "Path": "/",
    "GroupName": "MyGroupCli",
    "GroupId": "AGPAXZSFF03LCQYEHNDLU",
    "Arn": "arn:aws:iam::535942231766:group/MyGroupCli",
    "CreateDate": "2025-12-27T14:26:04+00:00"
  }
}
```

6. List policies that mention EC2:

```
aws iam list-policies \
--query "Policies[?contains(PolicyName, 'EC2')].{Name:PolicyName}" \
--output text
```

```
@toobashafique065 [main] $ aws iam list-policies \
> --query "Policies[?contains(PolicyName, 'EC2')].{Name:PolicyName}" \
> --output text
AmazonEC2FullAccess
AmazonEC2ReadOnlyAccess
AmazonElasticMapReduceforEC2Role
AmazonEC2RoleforDataPipelineRole
AmazonEC2ContainerServiceforEC2Role
AmazonEC2ContainerServiceRole
AmazonEC2RoleforAWSCodeDeploy
AmazonEC2RoleforSSM
CloudWatchActionsEC2Access
AmazonEC2ContainerRegistryReadOnly
AmazonEC2ContainerRegistryPowerUser
AmazonEC2ContainerRegistryFullAccess
AmazonEC2ContainerServiceAutoscaleRole
AmazonEC2SpotFleetAutoscaleRole
AWSElasticBeanstalkCustomPlatformforEC2Role
AmazonEC2ContainerServiceEventsRole
AmazonEC2SpotFleetTaggingRole
AWSSEC2SpotServiceRolePolicy
AWSServiceRoleForEC2ScheduledInstances
AWSSEC2SpotFleetServiceRolePolicy
AWSApplicationAutoscalingEC2SpotFleetRequestPolicy
AWSSEC2FleetServiceRolePolicy
AWSAutoScalingPlansEC2AutoScalingPolicy
```

7. Get ARN for AmazonEC2FullAccess (example query):

```
aws iam list-policies --query
'Policies[?PolicyName=='AmazonEC2FullAccess'].{Name:PolicyName, ARN:Arn}'
--output table
```

```
@toobashafique065 [main] $ aws iam list-policies --query 'Policies[?PolicyName=='AmazonEC2FullAccess'].{Name:PolicyName, ARN:Arn}' --output table
+-----+-----+
| ListPolicies |
+-----+-----+
| ARN | Name |
+-----+-----+
| arn:aws:iam::aws:policy/AmazonEC2FullAccess | AmazonEC2FullAccess |
+-----+-----+
```

8. Attach policy to group (use the ARN you retrieved):

```
aws iam attach-group-policy \
--group-name MyGroupCli \
--policy-arn arn:aws:iam::aws:policy/EXAMPLEPolicyName
```

```
@toobashafique065 [main] $ aws iam attach-group-policy --group-name MyGroupCli --policy-arn arn:aws:iam::aws:policy/AmazonEC2FullAccess
```

9. List attached policies for the group:

```
aws iam list-attached-group-policies --group-name MyGroupCli
```

```
@toobashafique065 @ /workspaces/Lab9 (main) $ aws iam list-attached-group-policies --group-name MyGroupCli
{
  "AttachedPolicies": [
    {
      "PolicyName": "AmazonEC2FullAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AmazonEC2FullAccess"
    }
  ]
}
```

10. Create a console login profile for the user:

```
aws iam create-login-profile \
--user-name MyUserCli \
--password <PASSWORD_VALUE> \
--password-reset-required
```

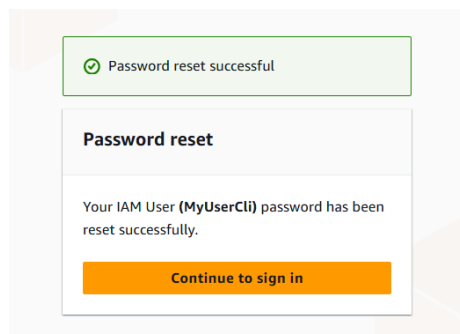
```
@toobashafique065 @ /workspaces/Lab9 (main) $ aws iam create-login-profile \
> --user-name MyUserCli \
> --password 'YourStrongPass123!' \
> --password-reset-required
{
  "LoginProfile": {
    "UserName": "MyUserCli",
    "CreateDate": "2025-12-27T14:51:01+00:00",
    "PasswordResetRequired": true
  }
}
```

11. If the user cannot change password, attach IAMUserChangePassword to the group temporarily:

```
aws iam attach-group-policy --group-name MyGroupCli --policy-arn
arn:aws:iam::aws:policy/IAMUserChangePassword
```

After the user logs in and resets password, detach that policy:

```
aws iam detach-group-policy --group-name MyGroupCli --policy-arn
arn:aws:iam::aws:policy/IAMUserChangePassword
```



```
@Nimra-Mursleen → /workspaces/Lab9 (main) $ aws iam detach-group-policy --group-name MyGroupCli --policy-arn
arn:aws:iam::aws:policy/IAMUserChangePassword
```

12. Create access keys for the user (save AccessKeyId and SecretAccessKey securely):

```
aws iam create-access-key --user-name MyUserCli
```

```
@toobashafique065 /workspaces/Lab9 (main) $ aws iam create-access-key --user-name MyUserCli
{
  "AccessKey": {
    "UserName": "MyUserCli",
    "AccessKeyId": "AKIAZSFFO3LAMWDIQ3D",
    "Status": "Active",
    "SecretAccessKey": "9GdMFE8yyv+LLNAAaS0RLwh3VkdikGymEWNUGaXC",
    "CreateDate": "2025-12-27T15:06:43+00:00"
  }
}
```

13. List access keys:

`aws iam list-access-keys --user-name MyUserCli`

```
@toobashafique065 /workspaces/Lab9 (main) $ aws iam list-access-keys --user-name MyUserCli
{
  "AccessKeyMetadata": [
    {
      "UserName": "MyUserCli",
      "AccessKeyId": "AKIAZSFFO3LAMWDIQ3D",
      "Status": "Active",
      "CreateDate": "2025-12-27T15:06:43+00:00"
    }
  ]
}
```

```
@toobashafique065 /workspaces/Lab9 (main) $ aws iam create-access-key --user-name MyUserCli
{
  "AccessKey": {
    "UserName": "MyUserCli",
    "AccessKeyId": "AKIAZSFFO3LAMWDIQ3D",
    "Status": "Active",
    "SecretAccessKey": "9GdMFE8yyv+LLNAAaS0RLwh3VkdikGymEWNUGaXC",
    "CreateDate": "2025-12-27T15:06:43+00:00"
  }
}
@toobashafique065 /workspaces/Lab9 (main) $ aws iam list-access-keys --user-name MyUserCli
{
  "AccessKeyMetadata": [
    {
      "UserName": "MyUserCli",
      "AccessKeyId": "AKIAZSFFO3LAMWDIQ3D",
      "Status": "Active",
      "CreateDate": "2025-12-27T15:06:43+00:00"
    }
  ]
}
```

14. (Don't) Delete access key:

`aws iam delete-access-key --user-name MyUserCli --access-key-id <AccessKeyId>` # Don't run this command

```
@toobashafique065 /workspaces/Lab9 (main) $ export AWS_ACCESS_KEY_ID=AKIAZSFFO3LESE7HION
@toobashafique065 /workspaces/Lab9 (main) $ export AWS_SECRET_ACCESS_KEY=9GdMFE8yyv+LLNAAaS0RLwh3VkdikGymEWNUGaXC
@toobashafique065 /workspaces/Lab9 (main) $ printenv | grep AWS_
AWS_SECRET_ACCESS_KEY=9GdMFE8yyv+LLNAAaS0RLwh3VkdikGymEWNUGaXC
AWS_ACCESS_KEY_ID=AKIAZSFFO3LESE7HION
@toobashafique065 /workspaces/Lab9 (main) $
@toobashafique065 /workspaces/Lab9 (main) $ aws iam get-user --user-name MyUserCli
An error occurred (AccessDenied) when calling the GetUser operation: User: arn:aws:iam::535942231766:user/MyUserCli is not authorized to perform the iam:GetUser action on resource: arn:aws:iam::535942231766:user/MyUserCli
```

15. After clearing or switching credentials, repeat get-user and save:

`exit` # to clear exports

`aws iam get-user --user-name MyUserCli`

```

@toobashafique065 [ ] /workspaces/Lab9 (main) $ exit
logout
Connection to localhost closed.
shell closed: exit status 254

C:\Users\malik\Lab9>gh codespace ssh -c fuzzy-bassoon-4jp7p65jj46p37pwv
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro
Last login: Sat Dec 27 13:54:33 2025 from ::1
@toobashafique065 [ ] /workspaces/Lab9 (main) $ aws iam get-user --user-name MyUserCli
{
  "User": {
    "Path": "/",
    "UserName": "MyUserCli",
    "UserId": "AIDAXZSFF03LFSOISHLCX",
    "Arn": "arn:aws:iam::535942231766:user/MyUserCli",
    "CreateDate": "2025-12-27T14:28:20+00:00",
    "PasswordLastUsed": "2025-12-27T14:58:27+00:00"
  }
}

```

Task 7 — Filters: query with filters to find instances and their attributes

Goal: Use filters and queries to list specific instances and attributes.

```

aws ec2 describe-instances \
--filters "Name=tag:Name,Values=MyServer" \
--query "Reservations[*].Instances[*].PublicIpAddress" \
--output text

```

```

@toobashafique065 [ ] /workspaces/Lab9 (main) $ aws ec2 describe-instances \
> --filters "Name=tag:Name,Values=MyServer" \
> --query "Reservations[*].Instances[*].PublicIpAddress" \
> --output text
40.172.111.55

```

```

aws ec2 describe-instances \

--filters "Name=instance-type,Values=t3.micro" \
--query "Reservations[].Instances[].InstanceId" \
--output table

```

```
@toobashafique065 [ ] /workspaces/Lab9 (main) $ aws ec2 describe-instances \
> --filters "Name=instance-type,Values=t3.micro" \
> --query "Reservations[*].Instances[*].InstanceId" \
> --output table
-----
| DescribeInstances |
+-----+
| i-0bfe79bb9bc36dba8 |
+-----+
```

```
aws ec2 describe-instances \
--filters "Name=subnet-id,Values=subnet-0600df5fa8ce60857" \
--query "Reservations[*].Instances[*].InstanceId" \
--output table
```

```
@toobashafique065 [ ] /workspaces/Lab9 (main) $ aws ec2 describe-instances \
> --filters "Name=subnet-id,Values=subnet-0d3e1389d968bb2ee" \
> --query "Reservations[*].Instances[*].InstanceId" \
> --output table
-----
| DescribeInstances |
+-----+
| i-0bfe79bb9bc36dba8 |
+-----+
```

```
aws ec2 describe-instances \
--filters "Name=vpc-id,Values=vpc-06be85cd81b657192" \
--query "Reservations[*].Instances[*].InstanceId" \
--output table
```

```
@toobashafique065 [ ] /workspaces/Lab9 (main) $ aws ec2 describe-instances \
> --filters "Name=vpc-id,Values=vpc-0b8fa1602193cc7f7" \
> --query "Reservations[*].Instances[*].InstanceId" \
> --output table
-----
| DescribeInstances |
+-----+
| i-0bfe79bb9bc36dba8 |
+-----+
```

Task 8 — Use --query to format outputs for reporting

Goal: Extract useful fields in table format for reporting.

```
aws ec2 describe-instances \
--filters "Name=tag:Name,Values=MyServer" \
--query
```

```
"Reservations[*].Instances[*].[InstanceId,PublicIpAddress,Tags[?Key=='Name'].Value|[0]]" \
--output table
```

```
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 describe-instances \
\
--query "Reservations[*].Instances[*].[InstanceId,PublicIpAddress,Tags[?Key=='Name'].Value|[0]]" \
--output table
> --filters "Name=tag:Name,Values=MyServer" \
> --query "Reservations[*].Instances[*].[InstanceId,PublicIpAddress,Tags[?Key=='Name'].Value|[0]]" \
> --output table
-----
|                               DescribeInstances                               |
|-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| i-0bfe79bb9bc36dba8 | 40.172.111.55 | MyServer |
|-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|
```

```
aws ec2 describe-instances \
--query "Reservations[*].Instances[*].[InstanceId,State.Name]" \
--output table
```

```
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 describe-instances \
> --query "Reservations[*].Instances[*].[InstanceId,State.Name]" \
> --output table
-----
|                               DescribeInstances                               |
|-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| i-0bfe79bb9bc36dba8 | running |
|-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|
```

```
aws ec2 describe-instances \

--query
"Reservations[*].Instances[*].[InstanceId,InstanceType,Placement.AvailabilityZone]" \
--output table
```

```
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 describe-instances \
> --query "Reservations[*].Instances[*].[InstanceId,InstanceType,Placement.AvailabilityZone]" \
> --output table
-----
|                               DescribeInstances                               |
|-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| i-0bfe79bb9bc36dba8 | t3.micro | me-central-1b |
|-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|
```

Cleanup — Remove resources to avoid charges

After verification, terminate and delete everything you created in AWS. Capture screenshots for each step.

1. Terminate instances:

```
aws ec2 terminate-instances --instance-ids i-EXAMPLE1234567890
```

```
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 terminate-instances --instance-ids i-0bfe79bb9bc36dba8
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-0bfe79bb9bc36dba8",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
```

2. Delete security group and key pair:

`aws ec2 delete-security-group --group-id sg-EXAMPLE1234567890`

`aws ec2 delete-key-pair --key-name MyED25519Key`

```
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 delete-security-group --group-id sg-sg-0f402fc7296bf67cb
An error occurred (InvalidGroupId.Malformed) when calling the DeleteSecurityGroup operation: The security-group ID 'sg-0f402fc7296bf67cb' is malformed
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 delete-security-group --group-id sg-0f402fc7296bf67cb
{
  "Return": true,
  "GroupId": "sg-0f402fc7296bf67cb"
}
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws ec2 delete-key-pair --key-name MyED25519Key
{
  "Return": true,
  "KeyPairId": "key-08db94fd170526b14"
}
```

3. Remove IAM users, access keys, groups:

`aws iam delete-access-key --user-name MyUserCli --access-key-id <AccessKeyId>`

`aws iam delete-login-profile --user-name MyUserCli`

`aws iam remove-user-from-group --user-name MyUserCli --group-name MyGroupCli`

`aws iam delete-user --user-name MyUserCli`

`aws iam detach-group-policy --group-name MyGroupCli --policy-arn`

`arn:aws:iam::aws:policy/EXAMPLEPolicyName`

`aws iam delete-group --group-name MyGroupCli`

```
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws iam delete-access-key --user-name MyUserCli --access-key-id AKIAZSFFO3LAMWDIQ3D
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws iam delete-access-key --user-name MyUserCli --access-key-id AKIAZSFFO3LESE7HION
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws iam list-access-keys --user-name MyUserCli
{
  "AccessKeyMetadata": []
}
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws iam delete-login-profile --user-name MyUserCli
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws iam remove-user-from-group --user-name MyUserCli --group-name MyGroupCli
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws iam delete-user --user-name MyUserCli
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws iam detach-group-policy --group-name MyGroupCli --policy-arn arn:aws:iam::aws:policy/AmazonEC2FullAccess
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws iam detach-group-policy \
> --group-name MyGroupCli \
> --policy-arn arn:aws:iam::aws:policy/AmazonEC2ReadOnlyAccess
@toobashafique065 [ /workspaces/Lab9 (main) ] $ aws iam delete-group --group-name MyGroupCli
```

4. Final verification (billing/resource groups)

```
@toobashafique065 /workspaces/Lab9 (main) $ aws ec2 describe-volumes --query "Volumes[].VolumeId" --output table
@toobashafique065 /workspaces/Lab9 (main) $ aws ec2 describe-snapshots --owner-ids self --query "Snapshots[].SnapshotId" --output table
@toobashafique065 /workspaces/Lab9 (main) $ aws ec2 describe-security-groups --query "SecurityGroups[].GroupName" --output table

aws: [ERROR]: argument operation: Found invalid choice 'ec2'

usage: aws [options] <command> <subcommand> [<subcommand> ...] [parameters]
To see help text, you can run:

    aws help
    aws <command> help
    aws <command> <subcommand> help

@toobashafique065 /workspaces/Lab9 (main) $ aws ec2 describe-security-groups --query "SecurityGroups[].GroupName" --output table
-----
|DescribeSecurityGroups|
+-----+
| default              |
+-----+
@toobashafique065 /workspaces/Lab9 (main) $ aws ec2 describe-key-pairs --query "KeyPairs[*].[KeyName]" --output table
@toobashafique065 /workspaces/Lab9 (main) $ aws iam list-groups --query "Groups[*].[GroupName]" --output table
@toobashafique065 /workspaces/Lab9 (main) $
```